



Nội dung

1. Kiến trúc phần mềm – Software architecture
2. Vai trò của kiến trúc phần mềm
3. Các góc nhìn kiến trúc – Architecture views
4. Các kiểu kiến trúc – Architectural patterns
5. Tài liệu thiết kế kiến trúc

2

Nội dung

1. Kiến trúc phần mềm – Software architecture
2. Vai trò của kiến trúc phần mềm
3. Các góc nhìn kiến trúc – Architecture views
4. Các kiểu kiến trúc – Architectural patterns
5. Tài liệu thiết kế kiến trúc

3

Kiến trúc phần mềm

- Các hệ thống thường được hợp thành từ các hệ thống con.
- Mục đích của việc thiết kế kiến trúc phần mềm là nhằm xác định:
 - Các hệ thống con (**subsystems**) cấu thành nên hệ thống cần xây dựng.
 - Các giao diện (**interfaces**) giữa các hệ thống con.
 - Các luật tương tác (**rules for interaction**) giữa các hệ thống con.

4

Kiến trúc phần mềm

- Trong quá trình thiết kế kiến trúc, người ta sẽ lựa chọn:
 - Sử dụng loại cơ sở dữ liệu nào?
 - Sử dụng loại server nào?
 - Sử dụng các thành phần bảo mật? Các middleware?
 - ...
- Thiết kế kiến trúc cũng là lúc người ta đánh giá một số thuộc tính của hệ thống một cách sớm nhất như:
 - Độ tin cậy (**reliability**)
 - Hiệu năng (**performance**)

5

Nội dung

1. Kiến trúc phần mềm – Software architecture
2. Vai trò của kiến trúc phần mềm
3. Các góc nhìn kiến trúc – Architecture views
4. Các kiểu kiến trúc – Architectural patterns
5. Tài liệu thiết kế kiến trúc

6

Vai trò của thiết kế kiến trúc phần mềm

- Kiến trúc phần mềm cung cấp một cái nhìn tổng quan về các thành phần con cấu thành nên một hệ thống và cách chúng tương tác, phối hợp với nhau.
- Kiến trúc phần mềm cung cấp các góc nhìn khác nhau về hệ thống cần xây dựng:
 - Góc nhìn thể hiện các thành phần (component) và mối liên hệ giữa chúng (về khía cạnh logic)
 - Góc nhìn về sự phân bố các thành phần lên các tài nguyên hệ thống (về khía cạnh vật lý)
 - Góc nhìn chi tiết về các mô-đun và mối liên hệ giữa chúng
- Trong quá trình phát triển phần mềm, việc thiết kế kiến trúc là rất quan trọng vì những lợi ích sau:

7

Vai trò của thiết kế kiến trúc phần mềm

- **Là phương tiện giao tiếp để truyền đạt ý tưởng, đề xuất (Understanding and communication)**
 - Kiến trúc phần mềm là phương tiện giao tiếp chính để người thiết kế kiến trúc trao đổi với các bên liên quan về những cấu trúc hệ thống được đề xuất trước khi bắt tay vào xây dựng:
 - Với người dùng cuối (end-users)
 - Với khách hàng (customers)
 - Với các lập trình viên (developers), kiểm thử viên (testers)
 - ...

8

Vai trò của thiết kế kiến trúc phần mềm

- **Giúp xác định các thành phần tái sử dụng (Reuse)**
 - Kiến trúc phần mềm thể hiện được **những phần nào là cố định**, dùng chung được, **những phần nào là thay đổi**
 - Từ đó giúp xác định được các thành phần có thể tái sử dụng.
 - Ví dụ: Authentication module → dùng chung cho nhiều hệ thống

9

Vai trò của thiết kế kiến trúc phần mềm

- **Giúp dễ dàng phân chia công việc xây dựng hệ thống (Construction and Evolution)**
 - Một bản thiết kế kiến trúc phần mềm sẽ phân hệ thống thành các hệ thống con/các thành phần con → Đây là cơ sở để phân chia công việc cho các đội/các cá nhân thuận tiện hơn.
 - Các đội/các cá nhân có thể làm việc đồng thời trên các phần khác nhau của cùng một dự án.

10

Vai trò của thiết kế kiến trúc phần mềm

- **Giúp phân tích một số thuộc tính quan trọng (Analysis)**
 - Kiến trúc phần mềm giúp phân tích, dự đoán trước một số thuộc tính quan trọng của hệ thống một cách sớm nhất, trước khi bắt tay vào thực hiện xây dựng hệ thống:
 - Độ tin cậy (Reliability)
 - Hiệu suất (Performance)
 - Chi phí (Cost) ...
 - Điều đó giúp cho chúng ta có những điều chỉnh, lựa chọn giải pháp phù hợp nhất với nhu cầu khách hàng.

11

Vai trò của thiết kế kiến trúc phần mềm

- Ví dụ:
 - Một kiến trúc có **Load Balancing** sẽ đạt hiệu quả và độ tin cậy cao hơn so với kiến trúc không có Load Balancing.
 - Kiến trúc có **tường lửa** sẽ có độ tin cậy cao hơn.
 - Kiến trúc có **cache** (ví dụ cache hình ảnh) sẽ có hiệu quả hơn.

12

Nội dung

1. Kiến trúc phần mềm – Software architecture
2. Vai trò của kiến trúc phần mềm
3. Các góc nhìn kiến trúc – Architecture views
4. Các kiểu kiến trúc – Architectural patterns
5. Tài liệu thiết kế kiến trúc

13

Các góc nhìn kiến trúc Architecture Views

- Trong lĩnh vực phần mềm, những góc nhìn kiến trúc khác nhau sẽ thể hiện những khía cạnh khác nhau về hệ thống phần mềm cần xây dựng.



Ví dụ, để xây dựng một tòa nhà, các bản vẽ thuộc các góc nhìn khác nhau được sử dụng:

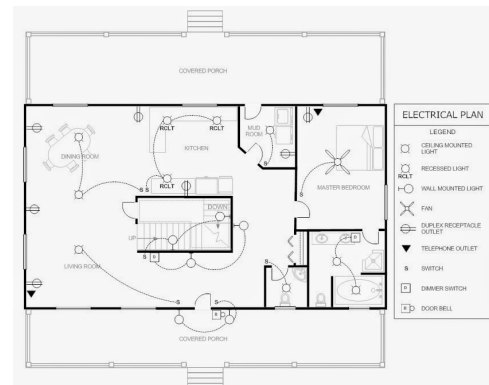
- Bản vẽ để thấy sơ đồ tòa nhà (floor plan)
- Bản vẽ phục vụ cho việc đi đường dây điện, đi ống nước ngầm
- Bản vẽ phục vụ cho vấn đề an toàn và phòng cháy chữa cháy

14



A house floor plan (a view of a architecture)

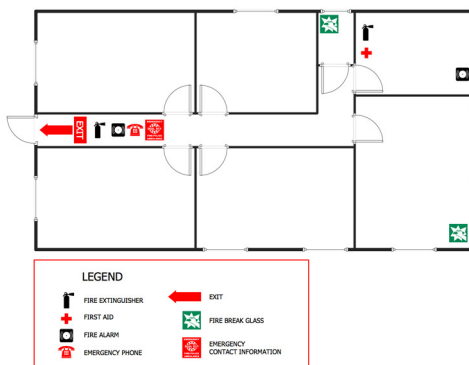
15



Electrical plan (a view of a architecture)

Src: <http://encodemaster.blogspot.com/2015/04/house-electrical-wiring-plan.html>

16



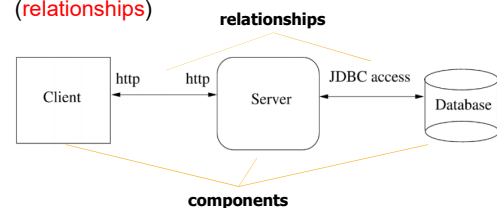
Office Emergency Plan (a view of a architecture)

Src: <https://www.conceptdraw.com/solution-park/building-fire-emergency-plans>

17

Các góc nhìn kiến trúc Architecture Views

- Một góc nhìn (a view) thể hiện cấu trúc của hệ thống:
 - Có những thành phần nào trong hệ thống (elements)
 - Mỗi quan hệ giữa các thành phần đó như thế nào (relationships)



18

Các góc nhìn kiến trúc Architecture Views

- Tác giả [Kruchten 1995] đề xuất kiến trúc gồm **4+1 views**:

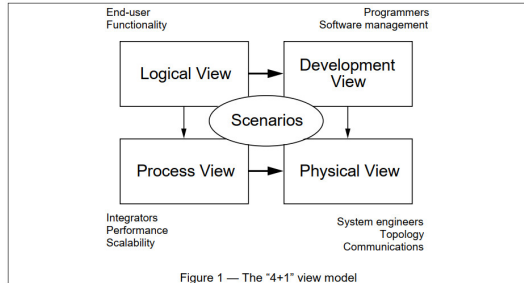


Figure 1 — The "4+1" view model

19

Các góc nhìn kiến trúc Architecture Views

- The **logical view**, which is the object model of the design.
 - UML diagrams: **Class, Object, State, Sequence, Communication** Diagram
- The **process view**, which captures the concurrency and synchronization aspects of the design
 - UML diagrams: **Activity** Diagram

20

Các góc nhìn kiến trúc Architecture Views

- The **physical view**, which describes the mapping(s) of the software onto the hardware and reflects its distributed aspect.
 - UML diagrams: **Component, Package** diagram
- The **development view**, which describes the static organization of the software in its development environment.
 - UML diagrams: **Deployment** diagram

21

Các góc nhìn kiến trúc Architecture Views

- Một quan điểm khác, theo [Pankaj 2008] tổng hợp, kiến trúc có thể chia thành 3 views sau:

Góc nhìn **Mô-đun**
Module view

Góc nhìn **Sự phân bổ**
Allocation view

Góc nhìn **Thành phần và Kết nối**
Component & Connector view

Cả 3 loại góc nhìn trên có thể được sử dụng để bổ sung cho nhau, cùng thể hiện cho cấu trúc của một hệ thống.

22

Architecture Views Module view

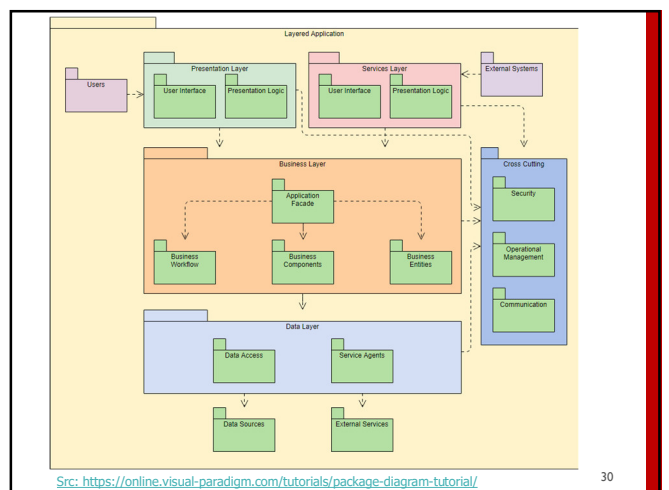
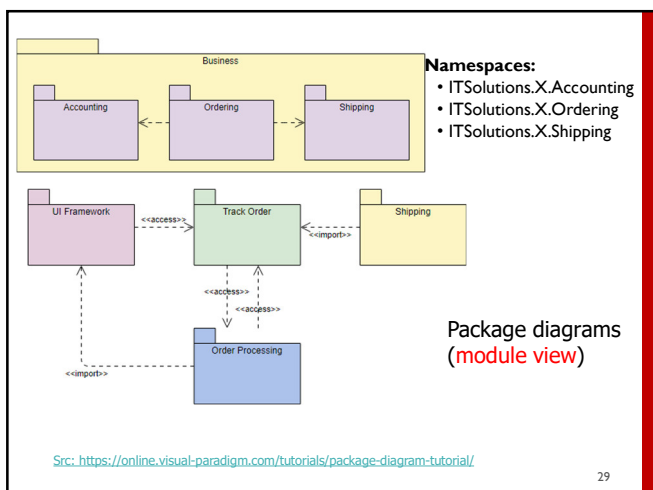
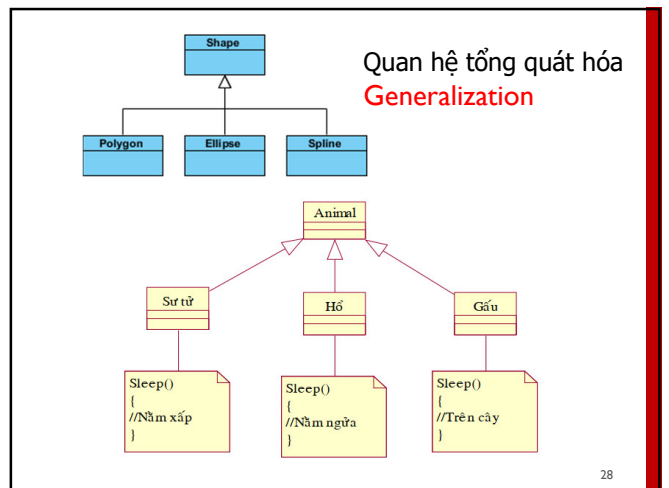
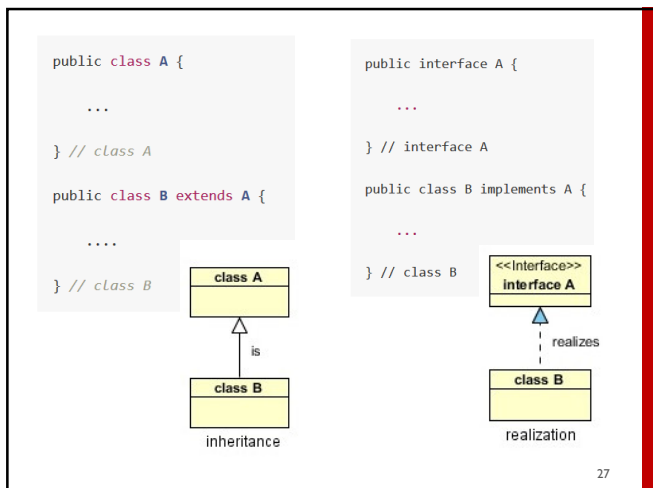
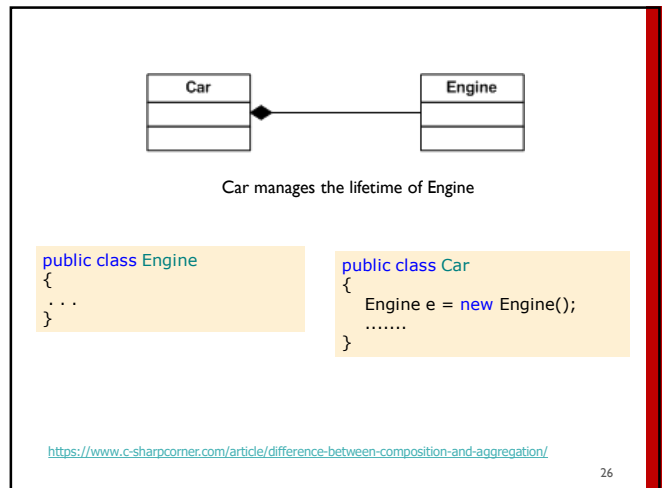
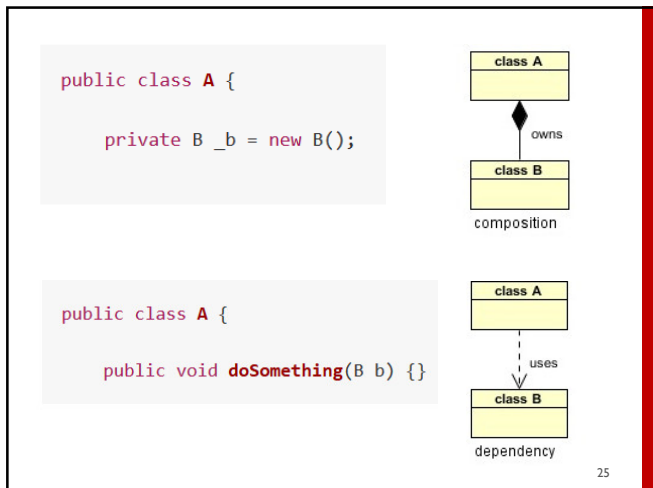
- In a module view, the system is viewed **as a collection of code units**, each implementing some part of the system functionality.
- The **main elements in this view are modules**.
- These views are code-based.
 - Examples of modules are **packages**, a **class**, a **procedure**, a **method**, a **collection of functions**, and a **collection of classes**.
- The relationships between these modules are also code-based and depend on how code of a module interacts with another module.

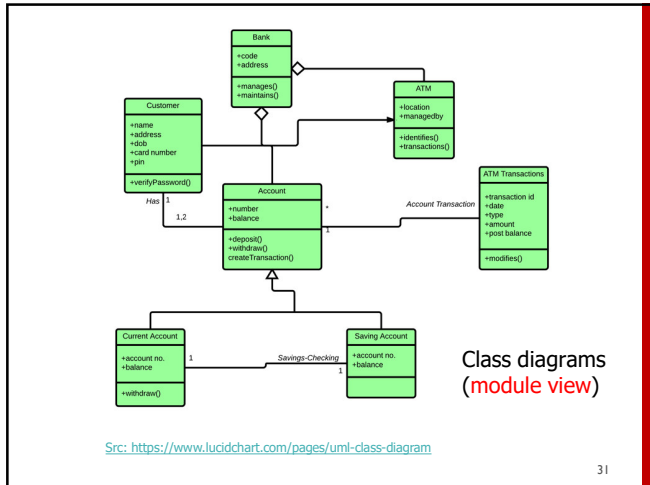
23

Architecture Views Module view

- Examples of relationships in this view are:
 - "**is a part of**" (i.e., module B is a part of module A)
 - "**uses or depends on**" (a module A uses services of module B to perform its own functions and correctness of module A depends on correctness of module B),
 - "**generalization or specialization**" (a module B is a generalization of a module A).

24





Architecture Views

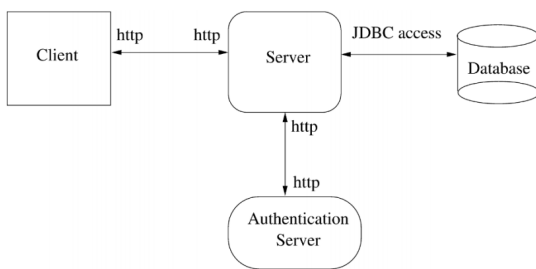
Component and connector view

- In a **component and connector (C&C) view**, the system is viewed as a collection of runtime entities called components. That is, a component is a unit which has an identity in the executing system.
- Objects (not classes), a collection of objects, and a process are examples of components.
- While executing, components need to interact with others to support the system services. Connectors provide means for this interaction.

32

Architecture Views

Component and connector view

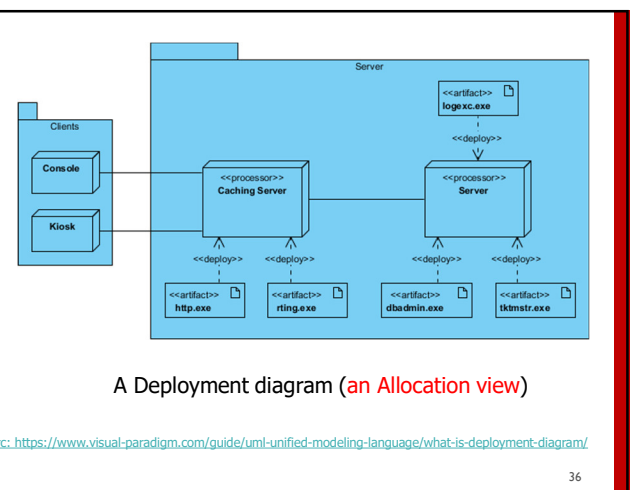
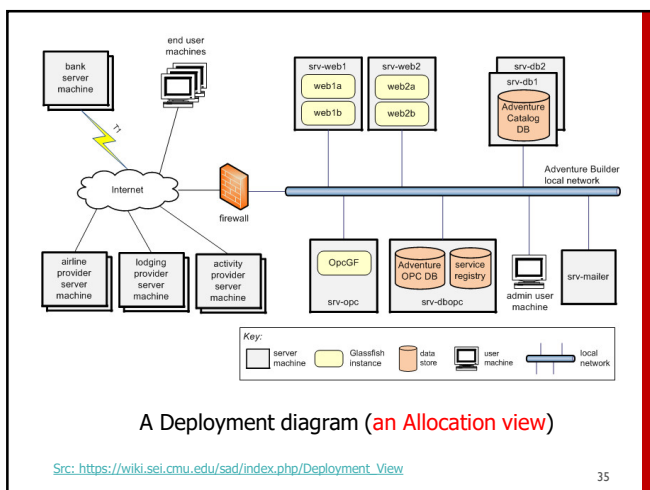


Architecture Views

Allocation view

- An **allocation view** focuses on how the different **software units** are **allocated** to **resources** like the hardware, file systems, and people.
- That is, an allocation view specifies the relationship between software elements and elements of the environments in which the software system is executed.

34



Architecture Views

Relationships between views

- The different views **represent the same system**. Hence, there are relationships between elements in one view and elements in another view.
- For example:
 - The relationship between modules and components may be one to one in that one module implements one component.
 - On the other hand, it may be quite complex with a module being used by multiple components, and a component using multiple modules.
- While creating the different views, the designers have to be aware of this relationship.
- **The C&C view** has become **the defacto primary view**, one which is almost always prepared when an architecture is designed.

37

Architecture and Design in software process

- Architecture is a design in that it is in the solution domain and talks about the structure of the proposed system.
- We can view **architecture** as a **very high-level design**, **focusing only on main components**, and the architecture activity as **the first step in design**.
- During **design lower-level**, issues like the **data structures, files, and sources of data**, have to be addressed, while such issues are not generally significant at the architecture level.
- We also take the view that **design** can be considered as **providing the module view** of the architecture of the system.

38

Component and Connector View

- The C&C architecture view of a system has two main elements:
 - **Components**: are usually computational elements or data stores.
 - **Connectors**: define the means of interaction between these components
- A C&C view describes a **runtime structure of the system** - what components exist when the system is executing and how they interact during the execution
- Most often when people **talk about the architecture**, they refer to the **C&C view**

39

Component and Connector View

Components

- Components are generally units of **computation** or **data stores** in the system.
- A component has a type, for example:
 - clients, servers, filters, controllers, actuators, and sensors

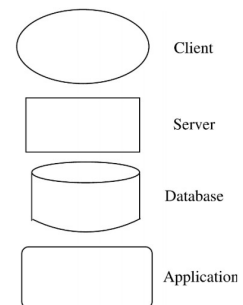


Figure 5.1: Component examples.

40

Component and Connector View

Connector

- Components provide **parts of** the services and features of the system, and these **must be combined** to deliver the overall system functionality.
- Interaction between components may be through a simple means:
 - For example, a component may interact with another using the **procedure call mechanism** (a connector), which is provided by the runtime environment for the programming language

41

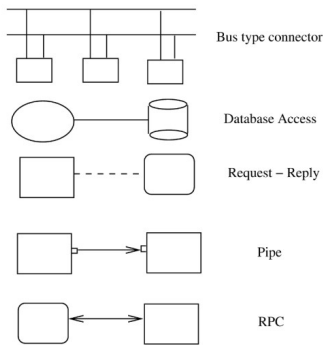
Component and Connector View

Connector

- The interaction may involve more complex mechanisms as well.
 - Examples of such mechanisms are **remote procedure call, TCP/IP ports**, and a protocol like **HTTP**
 - These mechanisms require a fair amount of **underlying runtime infrastructure**, as well as **special programming** within the components to use the infrastructure

42

Component and Connector View Connector



Note that connectors need **not be binary** and a connector may provide an **n-way communication** between multiple components.

- For example, a **broadcast bus** may be used as a connector, which allows a component to **broadcast its message to all the other components**.

Figure 5.2: Connector examples.

43

An example

- Suppose we have to design and build a simple system for taking an **on-line survey of students** on a campus.
- There is a set of multiple-choice questions, and the proposed system will provide the survey form to the student, who can fill and submit it on-line.
- We also want that when the user submits the form, he/she is also shown the current result of the survey, that is, what percentage of students so far have filled which options for the different questions.

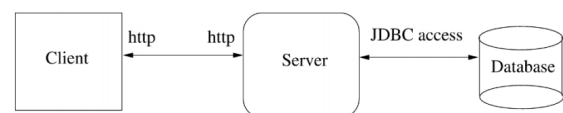
44

An example

- The system is best built using the Web technology.
- For this simple system, a traditional 3-tier architecture is proposed. It consists of:
 - Client component**: which will display the form that the student can complete and submit, and will also display the results.
 - Server component**: processes the data submitted by the student and saves it on the database.
 - Database component**: stores data

45

An example



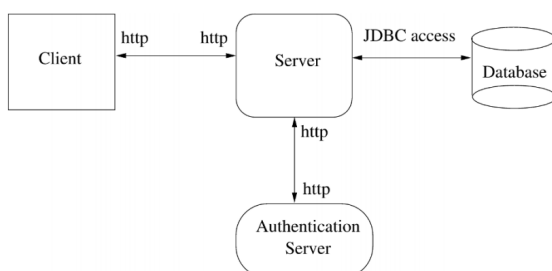
Note that at the architecture level, a host of details are not discussed:

- How is the URL** of the survey set?
- What are the modules** that go into building these components
- What language** are they written in? ...

46

Suppose this system be open only to registered students.

- To identify the students, it was explained that each student has an account, and their account information is available from the main proxy server of the institute.

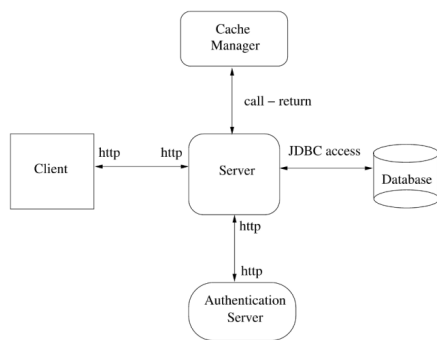


47

An example

- Now suppose we want the system to be extended in a different way.
- It was found that the database server is somewhat **unreliable**, and is **frequently down**.
- It was also felt that when the student is given the result of the survey when he submits the form, a **somewhat outdated result is acceptable**, as the results are really statistical data and a little inaccuracy will not matter.

48



If the database is **down or unavailable**, the survey **data is stored locally in a cache component**, and the **result saved in the cache component** is used to provide the result to the student.

49

Bài tập

- Thiết kế kiến trúc cho hệ thống website tin tức (ví dụ tương tự như vnexpress.net, ...)
- Hệ thống hiển thị tin tức
- Thông tin dự báo thời tiết
- Thông tin chứng khoán
- Giá vàng
- Có app trên thiết bị di động

50

Nội dung

1. Kiến trúc phần mềm – Software architecture
2. Vai trò của kiến trúc phần mềm
3. Các góc nhìn kiến trúc – Architecture views
4. Các kiểu kiến trúc – Architectural patterns
5. Tài liệu thiết kế kiến trúc

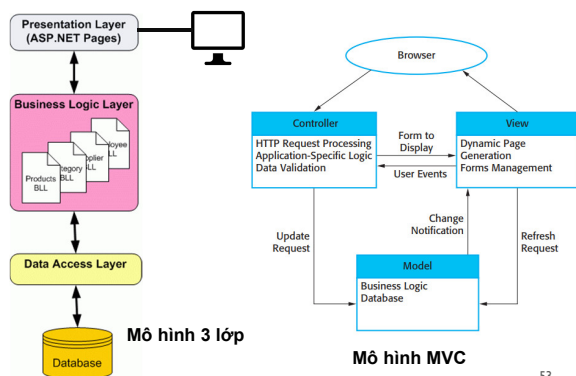
51

Các kiểu kiến trúc

- It should be clear that **different systems will have different architecture**.
- There are **some general architectures** that have been observed in many systems and that seem to represent general structures that are useful for architecture of a class of problems. These are called **architectural styles**.

52

Các kiểu kiến trúc Layered architecture



53

Các kiểu kiến trúc Layered architecture

- Description:
 - Organizes the system into **layers** with related functionality associated with each layer.
 - A layer **provides services** to the **layer above it**
 - so the **lowest-level layers** represent **core services** that are likely to be used throughout the system.

54

Các kiểu kiến trúc Layered architecture

• When used:

- Used when building new facilities on top of existing systems;
- When the development is spread across **several teams** with each team responsibility for a layer of functionality;
- When there is a requirement for multi-level security.

55

Các kiểu kiến trúc Layered architecture

• Advantages:

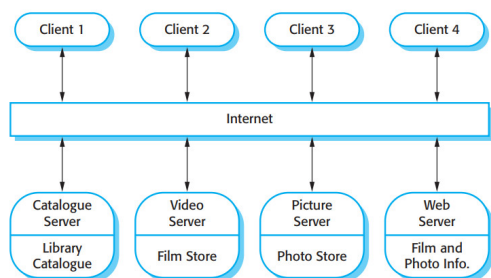
- Allows **replacement** of entire layers so long as the interface is maintained.

• Disadvantages:

- In practice, providing **a clean separation between layers is often difficult** and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it.
- **Performance** can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer

56

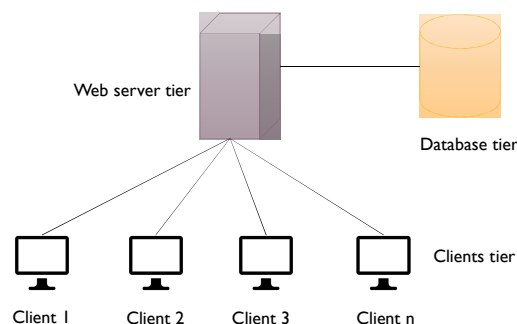
Các kiểu kiến trúc Client–server architecture



A client— server architecture for a film library

57

Các kiểu kiến trúc Client–server architecture



58

Các kiểu kiến trúc Client–server architecture

• Description:

- In a client–server architecture, the functionality of the system is organized **into services**, with each service delivered from a **separate server**.
- Clients are users of these services and access servers to make use of them.

• When used:

- Used when data in a shared database has to be **accessed from a range of locations**.
- Because servers can be **replicated**, may also be used when the load on a system is variable.

59

Các kiểu kiến trúc Client–server architecture

• Advantages:

- The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) **can be available to all clients** and does not need to be implemented by all services.

60

Các kiểu kiến trúc Client-server architecture

• Disadvantages:

- Each service is a single point of failure so susceptible to **denial of service attacks** or **server failure**.
- **Performance** may be **unpredictable** because it depends on the **network** as well as the system.
- May be **management problems** if servers are owned by **different organizations**.

61

Các kiểu kiến trúc Client-server architecture

• The major components of this model are:

- A **set of servers** that offer services to other components.
- Examples of servers include print servers that offer printing services, file servers that offer file management services, and a compile server, which offers programming language compilation services.

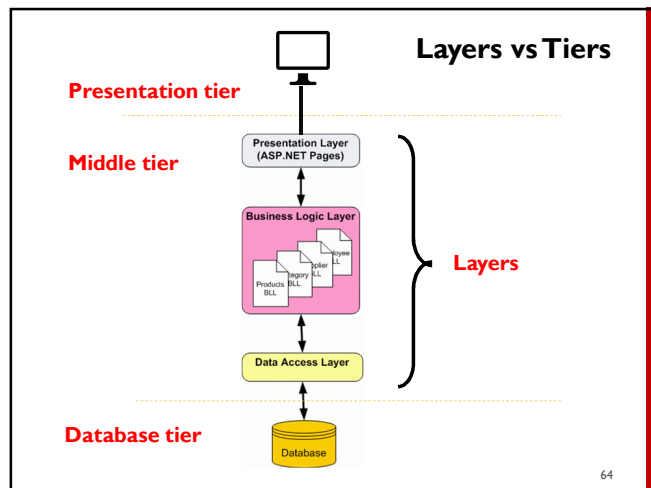
62

Các kiểu kiến trúc Client-server architecture

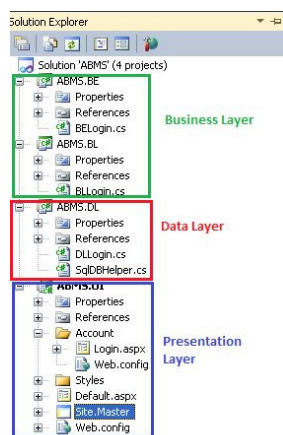
• The major components of this model are:

- A **set of clients** that call on the services offered by servers.
 - There will normally be several instances of a client program executing concurrently on different computers.
- A **network** that allows the clients to access these services.
 - Most client-server systems are implemented as distributed systems, connected using Internet protocols.

63



64



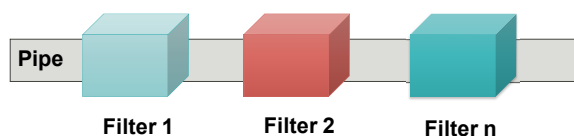
Layers
(in different namespaces)

65

Các kiểu kiến trúc Pipe and Filter

• The pipe-and-filter style has:

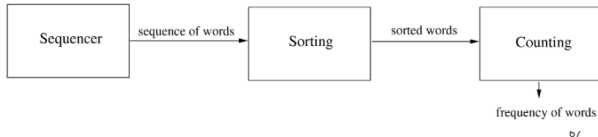
- **Component:** **Filters**
- **Connector:** **Pipe**
- Pipe-and-filter architectural style is well suited for **data processing and transformation**: text processing apps, signal processing apps



66

Các kiểu kiến trúc Pipe and Filter

- A filter **receives the data** it needs from some defined input pipes, performs the **data transformation**, and then **sends the output data** to other filters.
- A filter may have more than one input and more than one output.
- Filters can be **independent** and **asynchronous** entities.



Các kiểu kiến trúc Pipe and Filter

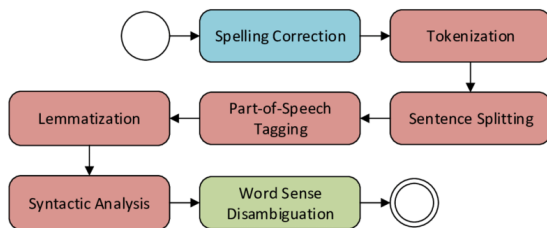
- A filter need not know the identity of the filter that sent the input data or the identity of the filter that will consume the data they produce.

```
public class SortEngine {
```

```
    public List<String> sort(String[] words) {
```

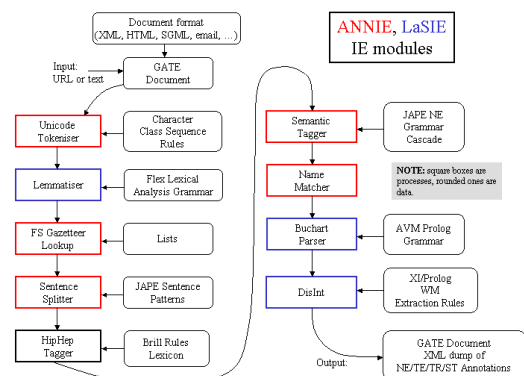


68



https://www.researchgate.net/figure/The-NLP-pipeline-used-at-the-basis-of-the-methods-features-compared-to-the-number-of_fig4_318138528

69



ANNIE components in a PIPE form

<https://gate.ac.uk/sale/tao/splitch6.html>

70

Các kiểu kiến trúc

• Một số kiểu kiến trúc phổ biến khác:

- Publish-Subscribe style
- Peer-to-peer style
- Shared-Data Style

71

Nội dung

1. Kiến trúc phần mềm – Software architecture
2. Vai trò của kiến trúc phần mềm
3. Các góc nhìn kiến trúc – Architecture views
4. Các kiểu kiến trúc – Architectural patterns
5. Tài liệu thiết kế kiến trúc

72

Documenting Architecture Design

- A document describing the architecture should contain the following:
 - **System and architecture context**
 - Description of **architecture views**
 - **Across views** documentation

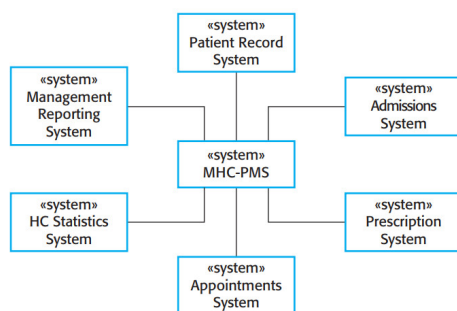
73

Documenting Architecture Design

- A context diagram that establishes the scope of the system, its boundaries, the key actors that interact with the system, and sources and sinks of data can also be very useful.
- A context diagram is frequently represented by showing the system in the center, and showing its connections with people and systems, including sources and sinks of data.

74

Documenting Architecture Design



A Context diagram

75

Documenting Architecture Design

- With the context defined, the document can proceed with **describing the different structures or views**.
- Multiple views of different types may be needed, and which views are chosen depends on the needs of the project and its stakeholders.
- The **description of views** in the architecture documentation contains:
 - pictorial representation of the view
 - supporting documentation for views

76

Combining of views

- In such situations, the different structures may look very similar and describing the views separately can lead to a repetition. In such situations, for practical reasons, it is better to combine different views into one.
- An example of combining is when there are multiple modules in the module view that form the different layers in the layer view.

77

Summary

- **Architecture** of a software system provides a **very high-level view** of the system in terms of parts of the system and how they are related to form the whole system.
- Architecture facilitates development of a high-quality system. It also allows **analysis** of many of the system properties like **performance** that depend mostly on architecture to be done early in the software life cycle.

78

Summary

- There are three main architectural views of a system—**module**, **component and connector**, and **allocation**.
 - In a **module view**, the system is viewed as a structure of programming modules like packages, classes, functions, etc.
 - In a **component and connector (C&C) view**, the system is a collection of runtime entities called **components**, which interact with each other through the **connectors**.
 - An **allocation view** describes how the different software units are **allocated** to **hardware resources** in the system.

79

Summary

- C&C view is most common, and is often the centerpiece of the architecture description.
- There are some common styles for a C&C view which have been found useful for creating this architecture view for a system.
 - These include pipe and filter, shared data, client-server, publish-subscribe, peer to peer, and communicating processes styles.
 - Each of these styles describes the types of components and connectors that exist and the constraints on how they are used.

80

Summary

- C&C view styles:
 - The pipe and filter has one type of component (filter) and one type of connector (pipe) and components can be connected through the pipe.
 - The client-server style has two types of components (client and server) and there is one connector (request/reply). A client can only communicate with the server, and an interaction is initiated by a client.
 - In shared data style the two component types are repository and data accessors. Data accessors read/write the repository and share information among themselves through the repository.

81

Tham khảo

- [1] Ian Sommerville, *Software Engineering*, 9th ed., Addison-Wasley, 2011.
- [2] Pankaj Jalote, *A Concise Introduction to Software Engineering*, Springer, 2008.

Đọc thêm:

- <https://edwardthienhoang.wordpress.com/software-architecture/>
- <https://herbertograca.com/2017/07/03/the-software-architecture-chronicles/>
- <https://sites.google.com/site/softwarearchitectureinpractice/>

82